

Dépouillement REAC 37827 "développeur IA" — Les compétences

cf. pour les remarques générales :

<https://docs.google.com/document/d/1UwhXSs8h8nO-uFmDzmNnbBqQ5ewJaYl49PXXKogjSgEc/edit?tab=t.0>

Note : ce document propose une commentaire du [REAC](#) (référentiel de compétence), il ne s'y substitue pas.

E1 : collecter, stocker et mettre à disposition des données

(Tout le bloc 1 est dans cet ensemble de compétences)

Le but est de récupérer des données de 5 formats différents via du code, de les cleaner, les harmoniser en DB, les agréger, et les exposer via une API sécurisée.

- Le temps sera limité sur E1 : vous présentez ici le projet + E1 lui-même, en 15 min. Ne prenez pas trop de temps sur l'intro, allez à l'essentiel
- On vous demande dans cet ensemble une **présentation complète du projet** : "acteurs, objectifs fonctionnels et techniques, environnement, contraintes techniques, budget, organisation du travail". Consacrez-y 2-3 min, et le reste aux compétences
- Vous devez prouver au jury que vous savez : scraper un site, parser du HTML, fusionner des données, et créer une API. Bref, **démontrer votre capacité à interagir avec les cinq types de sources**
- On vous demande du code versionné : il est fortement conseillé d'avoir un petit projet qui tourne pour E1. Vous pouvez mettre le lien du repo dans le rapport pour que le jury puisse y accéder en amont de la soutenance
- Le jury pourra poser des questions complémentaires : vous devez être capable d'expliquer le processus de traitement, les difficultés rencontrées, etc.
- Technos recommandées : Python, requests/BeautifulSoup (pour le scrapping), données en CSV, FastAPI, Postgres, sqlalchemy
- Identifiez un thème de données, puis des sources pour vos 5 sources (ex: le cinéma, des données sur l'eau...)
- Une fois les données collectées, faites l'agrégation d'au moins deux sources en SQL : soit via un JOIN, soit en concaténant deux tables
- Le choix des sources n'a pas d'importance, mais il faut que les données aient la même nature : si vous agrégez des ventes de voiture, les 5 sources doivent concerner des ventes de voitures. Critère : **toutes les données doivent pouvoir arriver dans une même table à l'arrivée.**
- Rapport : pas besoin de placer l'intégralité des scripts ni le rapport ou les annexe, seulement les parties particulièrement pertinentes pour démontrer les compétences

C1 : automatiser l'extraction des données

Vous devez automatiser la récupération de données issues de 5 sources différentes :

1. une Web API
2. du Web scraping
3. un fichier (CSV, JSON...)
4. une DB (SQL ou NoSQL)
5. un système big data (DuckDB, Spark, Hadoop, Cassandra...)

- Décrivez techniquement votre projet pour E1 : extraction -> retraitement -> agrégation -> sauvegarde des données brutes -> expo via API (techno/libs, services externes, langages)
- Identifier le thème commun de vos données, puis des sources, et en faire l'agrégation
- La certif est agnostique de techno, mais il faut pouvoir justifier globalement **en quoi les données demandent un traitement particulier** (volume ? non-structurée ?) et en quoi la techno que vous avez choisie est appropriée pour ce traitement
- Pour la source BigData, MongoDB ne répond pas vraiment aux critères des **trois V** (variété, volume, vitesse). Ne l'adoptez qu'en dernier recours, et préparez-vous à le défendre à l'oral
- Un critère de validation est que "le script est fonctionnel" (ça ne sera pas vérifié car il n'y a pas de démo dans E1) et versionné sur Git. Vous ne pouvez pas avoir un simple schéma décrivant une archi
- La "destination" évoquée est une DB, idéalement relationnelle (sauf si vous justifiez l'utilisation d'une base non-relationnelle)

Conseil : proposez un tableau récapitulatif de vos 5 sources :

- la source (API, scraping, fichier, DB, big data)
 - la provenance (Kaggle, data.gouv.fr, API interne de votre entreprise...)
 - le type de données (vente de grille-pains en Bulgarie en 2013, playlists...)
 - le format (HTML, CSV, JSON...)
 - la techno utilisée (requests + BS, pandas, pymongo)
 - le volume taille (en nb de lignes obtenues ou en taille de fichier)
 - les champs ("id, name, weight")
-
- Pour le scraping : mettez dans le rapport un screen de la page à scraper. Expliquez brièvement les packages BeautifulSoup et requests
 - Big data : essayez d'avoir un format dédié, comme Parquet
 - Votre script Python doit avoir une gestion d'erreur, même minimale, et sauvegarder les résultats en local dans une DB
 - Présentez rapidement le code pour le traitement des 5 sources (si vraiment vous n'avez pas les 5 sources, donnez à l'oral des exemples de sources manquantes, et comment vous vous y seriez pris
 - Il faut absolument 5 sources différentes pour valider le bloc (même si la plupart des systèmes réels en ont bcp moins...) Si vous n'avez rien à mettre dans un système big

data, importez-y des données tabulaires standard. L'important est de prouver votre maîtrise des technos associées

- Ça n'est pas grave si vous requêtez une source qui n'est pas utilisée plus tard dans l'agrégation
- Un screenshot de chaque données peut être intéressant

C2 : requêter des données en SQL, depuis une DB ou un système de big data

- On vous demande plusieurs requêtes SQL pour cette compétence (donc au moins 2)
- Attention, les données utilisées doivent vraiment venir de ces requêtes
- Ayez au moins une requête complexe avec un filtre, des conditions, une/des jointures
- Vous devez montrer une conscience minimale de l'enjeu d'optimisations (notion de coût de calcul d'une requête, index, requêtes n+1, grosse jointure à ne pas faire, éviter SELECT *...). Bonus : utilisez EXPLAIN/EXPLAIN ANALYZE !
- Vous pouvez parler très brièvement d'ORM ici si vous en utilisez
- Je vous conseille d'avoir un fichier par source :

/extrac/

/api.py

/csv.py

...

C3 : créer des règles d'agrégation de données issues de différentes sources

Vous devez prouver que vous savez harmoniser vos 5 sources en un seul jeu utilisable pour l'analyse.

- Faites apparaître clairement ce que vous harmonisez (unités ? float vs int ? âge vs date de naissance ?) : quel champ, quel format, etc.
- Vous devez avoir un script d'agrégation fonctionnel, versionné lui aussi sur Git, qui nettoie, agrège et normalise (si besoin), vous produire un seul jeu de données final
- Ce script doit être documenté un minimum : technos, langages, libs, etc. Vous devez être capable d'expliquer l'ordre et la logique de traitement
- Vous pouvez utiliser pandas pour l'agrégation plutôt que SQL, si vous justifiez (brièvement) votre choix

C4 : modéliser les données en DB, en respectant le RGPD

- À la fin, vous devez mettre toutes vos données dans une DB (nous conseillons une DB relationnelle, justifiez brièvement votre choix : Postgres, SQLite...)
- La modélisation des données doit suivre MERISE (MCD/MLD, clés primaires/étrangères, etc.)

- Comme pour l'extract, l'import des données doit être fonctionnel, le script versionné et documenté, etc. Cette partie a une faible valeur ajoutée, n'y consacrez pas des heures
- La partie RGPD est non-pertinente si vous n'avez pas de données perso. Tâchez alors de la mentionner brièvement : "Nous nous sommes assurés que...". Dites brièvement ce que vous feriez s'il y en avait. Parlez du registre des traitements + procédures de tri des données personnelle, pas besoin de détails

C5 : développer une API mettant à disposition le jeu de données

- Il vous faut maintenant exposer toutes vos données via une API REST
- Ayez toutes les routes classiques (liste, suppression, édition...) **dispos pour chaque ressource** (ex. : si vous avez une table planes et plane_brands, vous devrez avoir GET /planes, GET /planes/id, DELETE /planes/id, etc., puis la même chose pour /plane_brands/)
- On recommande d'utiliser FastAPI pour cette partie, qui a un OpenAPI/Swagger intégré (et donc une doc automatique des endpoints)
- On parle d'API "sécurisée", il faut donc authentifier les requêtes. Faites au plus simple
- Note : l'authentification peut être faite au niveau de l'API ou de la DB, tant que l'accès aux données demande une authentification

E2 : veille, choix et paramétrage d'un service d'IA

Vous devez partir d'un "besoin client" (en général fictif), et organiser une veille d'une techno ou d'un service qui parte de ce besoin. Vous devez présenter :

- votre dispositif de veille (idéalement automatisée),
- la problématique à traiter,
- un inventaire des services d'IA disponibles,
- une préconisation d'un ou plusieurs services,
- les étapes de config/install de ces services.

Quelques remarques :

- Il n'y a pas de démo attendue pour cette partie : contrairement à E1, 15' c'est très long, **il faut donc être assez détaillé cette fois-ci**
- Il est possible d'avoir une thématique commune au projet E3-E4 pour ce livrable, mais ça n'est pas obligatoire. **Nous conseillons d'en faire un mini-projet à part**, mais faites ce qui est le plus simple pour vous
- Il peut être intéressant d'automatiser votre veille technique...

C6 : réaliser une veille technique et réglementaire

- Vous êtes libres
- Bien dire "sur quoi" vous faites une veille : quel est le besoin du client ? (même si le besoin est fictif, spécifiez-le bien) Vous devez faire une veille sur un outil ou un service ! (service d'OCR, de TTS...)
- Beaucoup d'infos pertinentes sur l'IA viennent d'agrégateurs de news (Hacker News, TheNextWeb, etc.), de Twitter, et de blogs de développeurs. Il peut être intéressant de mettre en place une veille automatisée autour de ces sources (flux RSS, etc.). Vous pouvez ensuite utiliser des outils comme InoReader qui agrègent ces flux RSS
- Il n'y a pas besoin de passer des heures sur cette partie, prenez les items de la compétence un à un
- Il peut être intéressant de valider l'accessibilité à cet endroit (via le rapport de veille)

C7 : identifier des services d'IA préexistants

- Vous devez reformuler le besoin client en termes techniques, et faire un benchmark de solutions à partir de ce besoin reformulé
- Ce benchmark doit décrire précisément, sur un ensemble de critères, en quoi chaque outil/service correspond au besoin
- Identifiez bien les critères de choix qui justifient ce que vous conservez ou pas
- Vous devez parler ici d' "éco-responsabilité" : l'astuce consiste à transformer ça sous l'angle de la performance (outil plus performant = éco-responsable). Le REAC ne vous demande pas de prouver quoi que ce soit, juste de documenter : le service tourne-t-il sur un datacenter neutre en carbone ? y a-t-il un rapport d'impact ? la taille du modèle est-elle sobre pour la tâche ? Citez une ou deux dimensions concrètes (efficacité énergétique du modèle, engagements du fournisseur, possibilité d'un modèle plus petit à qualité égale)
- Vous devez apporter une conclusion claire en disant quel outil/service est retenu, et pourquoi

C8 : paramétrer un service d'IA

- Une fois cet outil choisi, il faut l'installer et le configurer
- Idéalement il est accessible via un service (par ex, si c'est un modèle d'OCR, vous pouvez créer un petit service avec FastAPI qui ne sera qu'un wrapper du modèle). Notez la mention du monitoring dans les critères de validation
- Vous devez documenter l'installation de votre service, profitez-en pour mentionner l'accessibilité !
- Vous n'êtes pas obligés d'avoir une auth, mais si ça n'est pas le cas, c'est mieux de justifier votre choix, même brièvement

E3 : intégrer, monitorer, déployer un service d'IA

Ceci est le versant “dev/ML” de votre projet chef-d'oeuvre/fil rouge. Il doit faire l'objet d'une démo. Votre objectif ici est de **montrer que vous savez utiliser un modèle de bout en bout dans une application** : packaging, monitoring, tests, deploy/CI.

Vous n'êtes pas obligé d'avoir un modèle local pour cet ensemble, ni même d'entraînement. Mais ça semble beaucoup plus simple d'en avoir. Dans Waterflow 2, nous proposons à la fois un modèle locale et une API externe.

C9 : développer une API exposant un modèle d'IA

- Cette compétence évalue la partie API, pas la partie modèle : on vérifie l'auth, les tests, la documentation
- Vous devez donc avoir une auth et des tests sur cette partie
- Montrer que vous connaissez le cadre OWASP, mais pas besoin d'y passer des heures
- Vous devez avoir des tests, même basiques, sur les endpoints de votre API. Pas d'interprétation à apporter puisque vos tests sont censés être green.

C10 : intégrer l'API d'un modèle ou d'un service d'IA dans une application

- Attention, dans C9 VOUS développez une API, mais ici vous intégrez une API tierce
-
- Encore des tests ici, mais spécifiquement sur le service que vous intégrez
- Attention, il faut proposer un renouvellement de l'authentification

C11 : déploiement/MLOps : Monitorer un modèle d'IA

- On attend ici un monitoring de modèle type mlflow, pas un monitoring applicatif comme dans C20
 - Si votre modèle est un LLM, vous êtes libres de vos métriques : TTFT, latence, débit, méthode RED (Rate, Errors, Duration), golden signal...
 - Attention à la confusion "monitoring de modèle" vs "monitoring d'appli" : la mention de kibana dans le REAC est une erreur ici
- L'environnement "bac à sable" est a priori en local “La chaîne de monitoring est d'abord testée dans un environnement sandbox” : ne veut rien dire ?
- Le mieux est un dashboard type Dash/Streamlit, mais au pire celui de mlflow est suffisant

C12 : déploiement/MLOps : Programmer les tests automatisés d'un modèle d'IA

- ne parler des tests que dans C12
- intéressant de justifier le taux de couverture retenu

C13 : déploiement/MLOps : Développer la CI d'un modèle d'IA

- pas besoin que l'entraînement du modèle déclenche un déploiement de celui-ci

E4 : gestion de projet

- **E3 et E4 sont deux versants d'un même projet** : E3 utilise des notions de dev et data science (dev API, entraînement d'un modèle, etc.), E4 les concepts de gestion de projet (analyse de besoin, cahier des charges, user stories...) et de CI (tests, deploy...)
- Vous n'êtes pas obligé d'utiliser des containers pour ce projet
- Attention à la répétition dans les critères entre tests et CI
- C14/C15/C16 : ce sont ici des "soft" skills de gestion de projet. Le projet technique est ici le prétexte pour développer ces compétences. Parlez d'Agile, de kanban, de user stories, etc.

C14 : analyser le besoin d'un commanditaire d'une application intégrant un service d'IA

- Cf. remarques générales sur l'a11n

C15 : à partir de cette analyse de besoin, concevoir le cadre technique de l'application

- Présentez un diagramme qui représente le trajet des données, sur n'importe au moins un diagramme d'étapes

C16 : coordonner sa réalisation technique

- La gestion de projet en mode Agile est obligatoire. Partez du besoin d'un projet fictif ou d'une expérience réelle en entreprise
- Ne passez pas une éternité à créer une orga fictive.
- Si vous avez des éléments de votre alternance, c'est cool : extrayez-en le plus d'éléments et partez de ça : ça rendra votre projet plus vivant.
- C'est l'organisation collective du projet et si possible l'agilité qui intéresse ici

C17 : développer les composants techniques et les interfaces

- C'est le gros bloc, très fourre-tout : prévoir plus de temps pour celui-là
- La gestion des droits d'accès doit se faire aussi au niveau du site utilisateur de l'application
- Idéalement les maquettes sont externes, mais ce n'est pas à vous de les faire, elles sont présumées. Quitte à ce que ce soit artificiel, mettez au moins un mini CDC, pour montrer que c'est un minimum maîtrisé ("je veux que mon appli respecte tel critère .." pour montrer que les critères sont validés)
- Utilisez au moins une maquette et montrez que vous y adhérez dans votre application. Au pire, faites une capture de votre appli et déclarez-la comme maquette
- Vous pouvez renvoyer aux parties précédentes pour éviter les redites

C18 : automatiser les phases de tests lors du versionnement

- Il y a des tests évoqués en C9, C10, C11, C12, C13 et C17. Cette partie concentre la partie CI/CD, on montre ici qu'on peut les automatiser
-

C19 : créer un processus de livraison continue

- RAS, déploiement qui intègre les tests de C18

E5 : monitoring et résolution de bug

Le but de cet ensemble est de construire une application fictive monitorée (C20), présentant au moins un bug (C21). Vous devrez ensuite résoudre ce bug, et documenter sa résolution.

E5 se fait a priori dans un projet à part, mais vous pouvez garder une thématique commune à E3/E4 pour ce livrable, car il y a peu d'intérêt à faire du monitoring/détection de bug dans un projet vide.

C20 : monitorer une application d'IA

Vous devez ici proposer une documentation technique du monitoring. L'idée est de mettre en place le monitoring et le log d'une application, pour pouvoir détecter et remonter automatiquement les incidents :

- Vos outils (collecteurs, logs, agrégateurs, filtres, dashboard, etc.) doivent être installés et opérationnels en local. Vos alertes sont configurées et fonctionnelles, les seuils sont définis (pas besoin d'en avoir 50, juste quelques uns)
- Le top est de mettre en place une stack de monitoring type ELK. Il vous faut juste un/des collecteurs, un agrégateur (Prometheus a priori), du log, et un dashboard (Grafana a priori)
- Il vous faut du log intégré au code de votre app. Minimum : ``import logging``. Bonus : Prometheus Loki. Super-bonus : stack ELK avec les 3 containers
- Il vous faut des alertes qui se déclenchent en fonction de différents seuils
- Attention aux solutions de monitoring "clé en main" compliquées, faites au plus simple
- Docker Compose est idéal pour ce projet. Vous devriez avoir en gros : un fichier ``docker-compose.yml``, un fichier ``prometheus.yml``, et un fichier ``dashboard.json`` pour configurer Grafana.
- Tests, monitoring et CI/CD doivent être implémentés et automatisés

Dans la doc :

- Définissez des métriques de monitoring, des seuils d'alerte pour chacune, et le choix de vos alertes (email, SMS, Slack...)
- Justifier le choix de vos outils de consolidation/suivi des indicateurs, et les outils de log
- Documentez l'installation et configuration de ces outils et leurs dépendances
- La documentation respecte l'accessibilité blabla

- attention à la confusion possible entre monitoring du modèle (C11) et monitoring du système (C20) : la mention de "feedback loop" dans C20 est une erreur

- les logs doivent surveiller des métriques : justifiez pourquoi on conserve/monitore tels ou tels logs

- montrez votre maîtrise du principe plutôt que des technos spécifiques

- intéressant de collecter du feedback utilisateur qui déclenche/remette en cause la pertinence du modèle, surtout si vous expliquez votre démarche

- pas d'obligation de mise en place de la réinjection

C21 : résoudre un incident technique et documenter la solution

On teste ici votre mise en place du monitoring applicatif via la résolution d'un bug. Partez d'une appli existante et **introduisez-y une erreur, que vous corrigerez ensuite**.

- Vous devez avoir un bug détectable par un outil automatisé (par ex. des tests, un `assert()`...) ou un log, et que vous pouvez corriger avec du code.

- Vous devrez reproduire le bug, l'analyser, coder un fix, tester votre solution, documenter le problème, et déployer la solution : le plus simple est de montrer votre commit Git (le code ET le commentaire)
- Idéalement vous avez à la fin un test qui passe pour prouver la résolution
- Votre explication est orale, mais vous devez montrer le tests, la branche de fix, le merge.
- Vous devez montrer le déploiement de votre solution via une CI (on conseille d'utiliser GitHub Actions pour la CI)
- Vous n'avez pas besoin en revanche de coder le fix en live, juste de pouvoir en parler
- Cette partie n'est pas très lourde, vous pouvez la faire à la fin
- Vous pouvez utiliser un LLM pour des idées de bug sur votre code